Day-Ahead Flight Delay Risk Prediction with Leakage-Free Features and Calibrated Classifiers

Atanu Adhikari, Sambhav Kumar, Nishant Ranjan Singh,
Sankalp M Tellur, Prince Kumar Singh
Advanced Machine Learning Course Project — Phase 1

***Abstract*—We predict, on the night before departure, whether a flight leaving New York City will arrive 15 or more minutes late. Using the 2013 NYC flights data (336,776 departures), we build schedule, calendar, congestion and historical-delay features that are either known from the published schedule or computed from strictly earlier dates, and we verify this with an automated leakage test. Logistic regression and XGBoost are tuned on October–November and tested once on December. On December both models clearly beat a historical route-rate baseline (PR-AUC 0.461 and 0.468 vs. 0.381; base rate 0.335), but absolute discrimination stays modest (ROC-AUC $\approx$ 0.64) because day-level weather is not observable a day ahead. We show that a calibrator fitted on the calm validation months under-predicts the much worse December (ECE rises from 0.073 to 0.102 for XGBoost) and that a rolling 28-day Platt recalibration fixes most of this (ECE 0.047). Finally, an offline what-if rule that moves high-risk flights by up to two hours finds only small predicted gains, mostly from moving flights earlier in the day.**

***Index Terms*—flight delay prediction, gradient boosting, logistic regression, probability calibration, data leakage, temporal validation**

I. Introduction

Flight delays are costly for airlines and passengers; a study commissioned by the US FAA estimated the total annual cost of delays in the United States in the tens of billions of dollars [1]. A delay risk score that is available *before* the day of operation could support decisions such as adding schedule buffer, re-timing a flight or warning passengers with tight connections.

Many published delay models use information that is only known close to or after departure, such as the departure delay itself or observed weather, and evaluate on random train/test splits [2]. Both choices make the problem easier than it is in practice. In this project we fix the decision time to the night before the flight and enforce it strictly. Our contributions are:

- a leakage-free feature pipeline (schedule, calendar, congestion, smoothed long-run and recent delay history) with an automated test that fails if any feature depends on same-day or future outcomes;
- a comparison of logistic regression (LR) and XGBoost under a chronological split, with PR-AUC, ROC-AUC, recall at a fixed precision and calibration;
- an analysis of calibration under the base-rate shift between validation and test months, and a rolling recalibration that adapts to it;
- an offline what-if rescheduling analysis with explicit assumptions.

II. Literature Review

Delay prediction. Sternberg et al. [2] review flight delay prediction and group the methods into statistical analysis, probabilistic models, network representation, operational research and machine learning; they note that results are hard to compare because studies differ in scope, delay definition and data. Rebollo and Balakrishnan [3] predict delays on US routes with random forests, using network delay states as features, and show that the predictive power drops as the prediction horizon grows, which is consistent with our day-ahead setting being hard. Choi et al. [4] combine on-time performance data with weather observations and compare decision trees, random forests, AdaBoost and k-NN, using sampling to handle class imbalance. Gui et al. [5] combine flight, weather and ADS-B data and compare LSTM and random forest models. Yazdi et al. [6] propose a deep network trained with the Levenberg–Marquardt algorithm on US flight data. Most of these works use observed weather or near-departure network state, which is not available at our decision time.

Delay propagation. Pyrgiotis et al. [7] model how delays propagate through an airport network via aircraft rotations. This motivates our features on the aircraft's position in its daily rotation and on recent airport delay levels, and it explains the strong time-of-day effect in our data.

Models. Gradient boosting [8] fits an additive model of regression trees by functional gradient descent; XGBoost [9] adds a second-order approximation of the loss and explicit regularisation of the tree structure. Logistic regression remains a strong baseline for tabular risk models and is the AML baseline required by the project brief.

Calibration. For a risk score to be used in decisions, its probabilities should be calibrated. Platt scaling [10] fits a sigmoid on the model score; isotonic regression [11] fits a monotone step function. Niculescu-Mizil and Caruana [12] show that boosted trees are often poorly calibrated and benefit from these corrections, and Guo et al. [13] popularised the expected calibration error (ECE) and reliability diagrams. When the class prior changes between training and deployment (label shift), the posterior must be adjusted [14], [15]; more generally, models in non-stationary settings need to adapt to drift [16].

Evaluation and leakage. With imbalanced classes the precision–recall curve is more informative than ROC [17]. Kaufman et al. [18] formalise leakage as the use of information

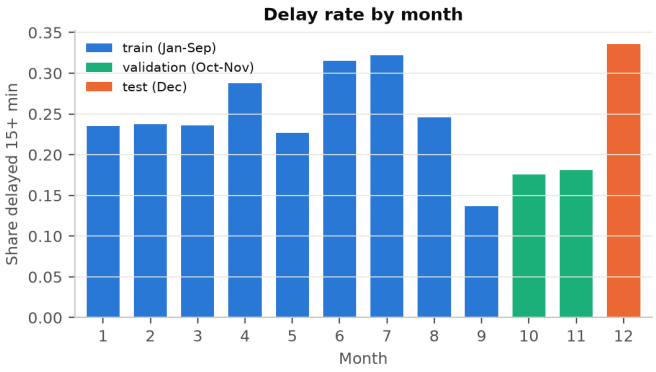


Fig. 1. Share of flights delayed 15+ minutes per month, coloured by split.

that would not be available at prediction time, which is the main pitfall in delay prediction.

Gap. Compared to prior work, we restrict ourselves to strictly day-ahead information, verify that restriction with a test, split chronologically, and report calibration under temporal shift rather than only ranking metrics.

III. Dataset and Exploratory Analysis

We use the Kaggle copy of `nycflights13` [19]: all 336,776 domestic departures from EWR, JFK and LGA in 2013, operated by 16 carriers to 105 destinations with 4,043 aircraft. The data come from the BTS on-time performance reporting [20].

Quality. 8,255 flights were cancelled (no departure time) and 1,175 departed but have no arrival delay (diverted). These 9,430 flights cannot be labelled, so they are excluded as examples but kept when counting scheduled volume, since the schedule is known in advance. 2,512 rows have no tail number; there are no duplicate ids.

Target. A flight is *delayed* if `arr_delay` ≥ 15 minutes, the BTS definition. 24.5% of labelled flights are delayed. The delay distribution is strongly right-skewed (median -5 minutes, maximum 1,272 minutes).

Leakage. Departure delay and arrival delay have a correlation of 0.915, so any post-departure field (`dep_time`, `dep_delay`, `arr_time`, `air_time`) would leak the label; all are removed.

Seasonality and split. Fig. 1 shows large month-to-month variation, from 13.7% delayed in September to 33.5% in December. Our chronological split trains on January–September, validates on October–November (base rate 0.178) and tests on December (0.335). The test month is therefore the worst month of the year and the validation months among the calmest, a shift that drives several of our findings.

Time of day and persistence. The delay rate rises steadily from 10.5% for 05:00 departures to 37% at 21:00 (Fig. 2, left), the signature of delay propagation. Saturday is the best day (17.5%) and Thursday the worst (28.5%). The daily delay rate has a lag-1 autocorrelation of 0.40 that drops to 0.13 at lag 2 (Fig. 2, right): disruption carries over to the next day but fades quickly. Carriers with at least 1,000 flights range from about 19% to 35% delayed, and many routes and flight numbers have few observations, which motivates smoothing of historical rates.

TABLE I
Feature Groups

Group	Features
Categorical	carrier, origin, destination
Calendar	sin/cos of scheduled departure and arrival time of day, sin/cos of day of week, weekend flag, days to nearest US federal holiday
Schedule	departure hour, distance, overnight arrival flag
Congestion	scheduled departures per origin-day, per origin-hour, per destination-day, per carrier-day; aircraft's number of NYC departures that day and position of this leg
Long-run history	smoothed past delay rate per carrier, origin, destination, route, carrier-route, origin-hour, flight number, aircraft
Recent history	smoothed 7-day delay rate per origin, carrier, destination; previous day's system and origin delay rate, origin cancellation rate and mean arrival delay

IV. Problem Formulation

Let flight i be scheduled on date d_i with schedule attributes s_i (carrier, origin, destination, scheduled times, distance, aircraft). The decision is made at the end of day $d_i - 1$. The information set is

$$\mathcal{I}_i = \{s_j : d_j = d_i\} \cup \{(s_j, y_j) : d_j < d_i\}, \tag{1}$$

i.e. the full schedule of the day (published in advance) plus outcomes of all earlier days. We assume outcomes of day $d_i - 1$ are known at the decision time; a few late evening flights land after midnight, which makes this assumption slightly optimistic. The task is to estimate $P(y_i = 1 \mid \mathcal{I}_i)$ with $y_i = \mathbb{1}[\text{arr_delay}_i \geq 15]$.

V. Feature Engineering

All 35 features (3 categorical, 32 numeric) fall into five groups (Table I).

Cyclic encoding. Time of day t (minutes) is encoded as $(\sin 2\pi t/1440, \cos 2\pi t/1440)$ so that 23:59 and 00:00 are close.

Past-only aggregation. For a grouping key g (e.g. route) let $k_g(d)$ and $n_g(d)$ be the number of delayed and of labelled flights of g on day d. The history features for a flight on day d use only

$$K_g(d) = \sum_{d' < d} k_g(d'), \qquad N_g(d) = \sum_{d' < d} n_g(d'), \tag{2}$$

or the same sums restricted to $d - w \leq d' < d$ for a window of w days. In code this is a cumulative (or rolling) sum over a daily table followed by a one-day shift.

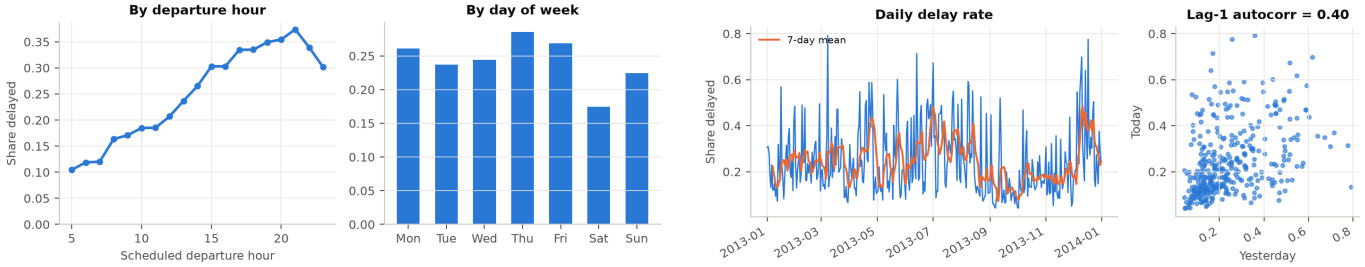


Fig. 2. Left: delay rate by scheduled departure hour and day of week. Right: daily delay rate over 2013 and yesterday-vs-today scatter.

Smoothing. Raw rates K/N are noisy for rare keys. We shrink them towards the system-wide past rate $p_0(d)$ with $m = 20$ pseudo-observations,

$$\hat{r}_g(d) = \frac{K_g(d) + m\,p_0(d)}{N_g(d) + m}, \tag{3}$$

which is the posterior mean under a $\mathrm{Beta}(mp_0, m(1-p_0))$ prior and the same idea as target encoding with smoothing [21].

Leakage test. The feature notebook builds the features, then replaces all outcomes on and after a cutoff date with random values and rebuilds them. Every feature on or before the cutoff must be unchanged. We also checked that the test fails when the one-day shift is removed.

Removed features. Day-of-year and month features were dropped: with one year of data, October–December never appear in training, so their seasonality cannot be learned, and in preliminary validation runs they only helped XGBoost memorise particular training days.

VI. Methods

A. Logistic Regression

LR models $p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$ and minimises the L2-regularised log-loss

$$\mathcal{L}(\mathbf{w}, b) = -\sum_i \big[y_i \log p_i + (1-y_i)\log(1-p_i)\big] + \frac{1}{2C}\|\mathbf{w}\|^2. \tag{4}$$

The objective is convex, so the solution is unique. At the optimum the gradient with respect to b vanishes, giving $\sum_i p_i = \sum_i y_i$: LR is calibrated in the large on its training data, so its raw probabilities are anchored to the January–September base rate (0.250) rather than to the calm validation months. Numeric features are median-imputed and standardised; categoricals are one-hot encoded (levels with fewer than 50 flights are pooled).

B. XGBoost

XGBoost [9] builds $F(\mathbf{x}) = \sum_k f_k(\mathbf{x})$ from regression trees and minimises $\sum_i \ell(y_i, F(\mathbf{x}_i)) + \sum_k \Omega(f_k)$ with $\Omega(f) = \gamma T + \frac{1}{2}\lambda\|\mathbf{w}\|^2$, where T is the number of leaves and $\mathbf{w}$ the leaf values. Each new tree uses a second-order expansion with $g_i = p_i - y_i$ and $h_i = p_i(1-p_i)$ for log-loss. For a leaf with instance set I_j, $G_j = \sum_{I_j} g_i$, $H_j = \sum_{I_j} h_i$, the optimal value is $w_j^* = -G_j/(H_j + \lambda)$ and a split gains

$$\frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma. \tag{5}$$

The `min_child_weight` parameter requires H_j above a threshold. Since $h_i \le 1/4$, a value of 2000 forces at least 8,000 flights into every leaf, about nine days of NYC departures. This turned out to be essential (Sec. VIII-B). Categoricals are handled natively.

C. Calibration

Platt scaling fits $q = \sigma(a \cdot \mathrm{logit}(p) + c)$ on held-out data; *isotonic regression* fits a non-decreasing step function by pool-adjacent-violators. Both are monotone, so ranking metrics do not change. Under pure label shift, Bayes' rule gives $\mathrm{logit}\, q = \mathrm{logit}\, p + \log\frac{\pi_t}{1-\pi_t} - \log\frac{\pi_s}{1-\pi_s}$ for source and target priors π_s, π_t [14]: the intercept c in Platt scaling is exactly the term that has to follow the base rate. We therefore also use *rolling* Platt scaling, refitted every day on the previous w days of out-of-sample scores and outcomes (all available at the decision time).

D. Metrics

PR-AUC (average precision) is the headline metric [17]; we also report ROC-AUC, Brier score $\frac{1}{N}\sum_i (p_i - y_i)^2$ and $\mathrm{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{N} |\bar{y}_{B_b} - \bar{p}_{B_b}|$ with ten equal-width bins [13]. As an operating point we require a precision of at least 0.5 (at least half of the flagged flights are really delayed) and report the recall achieved.

VII. Experimental Setup

The training, validation and test sets contain 244,737, 55,589 and 27,020 labelled flights. All choices use validation data only; December is scored once.

- **LR:** $C \in \{10^{-3}, \ldots, 10\}$, selected $C = 10$.
- **XGBoost:** learning rate 0.05, subsample and column sample 0.8, up to 3000 trees with early stopping (100 rounds) on validation PR-AUC; grid over depth $\{3, 4, 6\}$, `min_child_weight` $\{100, 500, 2000\}$, $\lambda \in \{1, 20\}$. Selected: depth 4, `min_child_weight` 2000, $\lambda = 20$, 308 trees.
- **Calibration:** Platt vs. isotonic chosen by fitting on October and scoring Brier on November and vice versa; Platt

TABLE II
DECEMBER TEST RESULTS (PLATT-CALIBRATED ON OCT–NOV)

Model	PR-AUC	ROC-AUC	Brier	ECE
Prior rate	0.335	0.500	0.230	0.086
Route history	0.381	0.554	0.230	0.095
LR	0.461	0.640	0.222	0.103
XGBoost	0.468	0.645	0.221	0.102

won for both models and was refitted on all of validation. The rolling window $w \in \{7, 14, 28\}$ was chosen by Brier score on November, giving $w = 28$ days for both models.

- **Threshold:** lowest threshold with validation precision ≥ 0.5.
- **Baselines:** constant training base rate, and the smoothed historical route delay rate as a score.

The code is organised as five Jupyter notebooks, one work package per team member (see Team Contributions), and uses scikit-learn [22] and XGBoost; running them in order reproduces every number and figure in this report. The code, results and this report are available at https://github.com/techatanu/Flight_Delay_Risk.

VIII. RESULTS

A. Discrimination

Table II and Fig. 3 summarise December performance. Both models clearly improve on the route-history baseline. XGBoost is slightly ahead of LR on December (PR-AUC 0.468 vs. 0.461), while LR was better on validation (0.297 vs. 0.276). The difference between the two models is small relative to the gap between validation and test periods.

B. Overfitting of XGBoost

With default-sized leaves, XGBoost fits the training months far better than it generalises: the selected model still reaches a training PR-AUC of 0.526 (ROC-AUC 0.752) against 0.276 (0.650) on validation, while LR goes from 0.452 (0.710) to 0.297 (0.664). (PR-AUC values are not directly comparable across periods because base rates differ, but ROC-AUC is.) Several features are constant within a day, such as the previous-day rates, so their combination acts as a fingerprint of the date; deep trees with small leaves use it to memorise the weather of individual training days. The validation grid preferred the largest `min_child_weight`, which forces each leaf to span many days.

C. Class Imbalance

About a quarter of training flights are delayed. We did not reweight the classes in the main models, and Table III checks that choice: the same models retrained with `class_weight="balanced"` (LR) and `scale_pos_weight` $= n_{\text{neg}}/n_{\text{pos}}$ (XGBoost), compared on raw probabilities. Weighting leaves PR-AUC and ROC-AUC practically unchanged but inflates every probability: the mean predicted risk rises to about 0.5 against an observed December rate of 0.335, and ECE more than doubles for both models.

TABLE III
EFFECT OF CLASS WEIGHTING (RAW PROBABILITIES)

Model	Val PR-AUC	Test PR-AUC	Test ROC-AUC	Test ECE	Mean $\hat{p}$
LR (unweighted)	0.297	0.461	0.640	0.064	0.295
LR, balanced class weights	0.296	0.460	0.641	0.182	0.517
XGBoost (unweighted)	0.276	0.468	0.645	0.073	0.269
XGBoost, positive weight 3.00	0.273	0.466	0.647	0.153	0.488

TABLE IV
OPERATING POINT CHOSEN ON VALIDATION (PRECISION ≥ 0.5), APPLIED TO DECEMBER

Model	Threshold	Flagged	Precision	Recall
LR	0.519	1.7%	0.612	0.031
XGBoost	0.498	1.8%	0.661	0.035
LR (rolling cal.)	0.648	0.9%	0.557	0.014
XGBoost (rolling cal.)	0.576	3.6%	0.585	0.062

Platt scaling could undo the shift, but then weighting would add nothing. We therefore handle the imbalance through the metric (PR-AUC) and the precision-based threshold instead. Oversampling such as SMOTE was not used, because synthetic flights interpolated between different days are inconsistent with the day-level history features.

D. Operating Point

Table IV applies the validation threshold to December. Precision stays above the target (0.661 for XGBoost), but recall is only 0.035: because December is much worse than October–November, the fixed calibration under-predicts and the threshold flags very few flights. For reference, the best recall at precision ≥ 0.5 that could be achieved on December (read off the test curve, so not attainable in practice) is 0.292 for XGBoost and 0.276 for LR. With rolling calibration the XGBoost threshold recovers some recall (0.062 at precision 0.585); for LR it does not (0.014 at precision 0.557).

E. Calibration

Table V and Fig. 4 compare raw, fixed and rolling Platt probabilities on December. Fixed calibration *worsens* ECE for both models (LR 0.064 $\rightarrow$ 0.103, XGBoost 0.073 $\rightarrow$ 0.102): it learned the low validation base rate and pulls the mean prediction down to 0.234 while the observed rate is 0.335. Rolling recalibration follows the shift and gives the best ECE (0.045 and 0.047) and the best Brier score for XGBoost. The cost is a lower pooled PR-AUC (0.450 for XGBoost), because each day now gets its own monotone map, which changes the ranking *across* days.

F. Feature Ablation and Importance

Table VI and Fig. 5 add one feature group at a time with the selected hyperparameters. Congestion features give the largest gain on validation. Recent-history features slightly hurt on the calm validation months but give the largest gain on December, where disruption carries over from day to day. This is consistent with the EDA: persistence matters on bad days and is noise on calm ones. Permutation importance [23] on validation (Fig. 6) ranks the scheduled daily volume at the

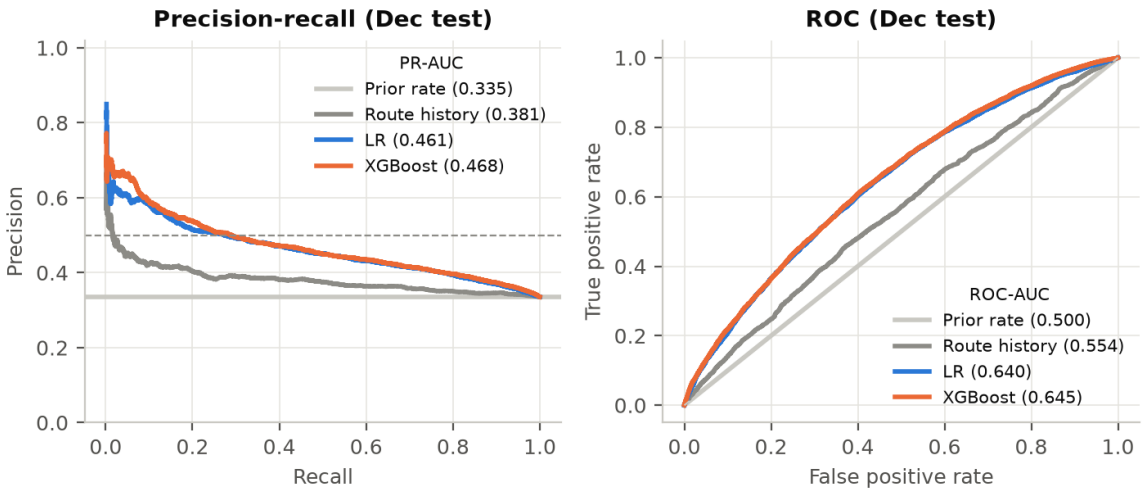


Fig. 3. Precision–recall and ROC curves on the December test set. Dashed line in the left panel: the 0.5 precision target.

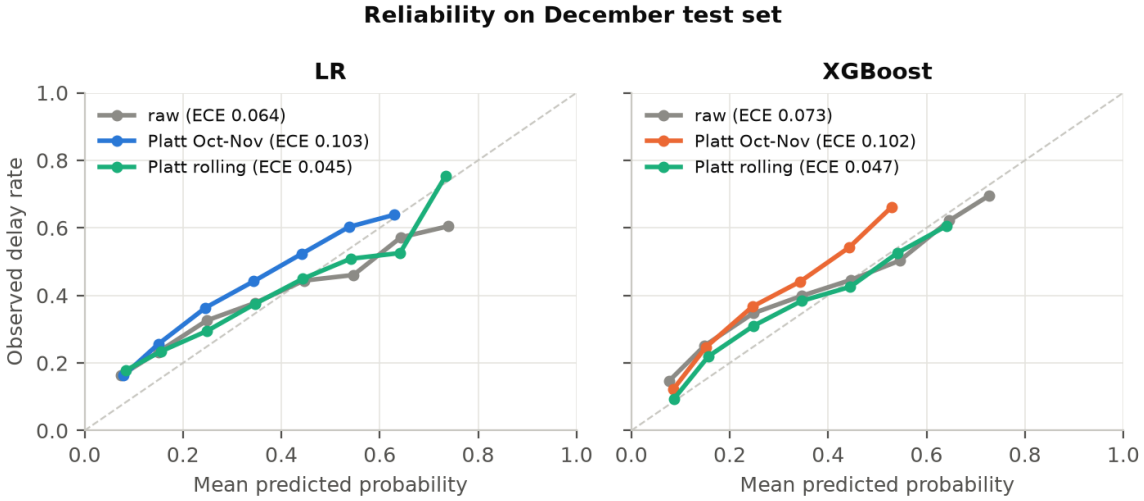


Fig. 4. Reliability diagrams on December (bins with fewer than 50 flights omitted).

TABLE V
CALIBRATION ON DECEMBER

Model	Probabilities	Brier	ECE	Mean $\hat{p}$
LR	raw	0.2159	0.0640	0.295
LR	Platt (Oct-Nov)	0.2218	0.1034	0.232
LR	Platt (rolling)	0.2168	0.0450	0.298
XGBoost	raw	0.2163	0.0734	0.269
XGBoost	Platt (Oct-Nov)	0.2205	0.1017	0.234
XGBoost	Platt (rolling)	0.2150	0.0474	0.298

TABLE VI
FEATURE-GROUP ABLATION (PR-AUC)

	Validation		Test	
Feature set	LR	XGB	LR	XGB
schedule + calendar	0.282	0.288	0.430	0.436
+ congestion	0.309	0.297	0.431	0.424
+ long-run history	0.307	0.296	0.440	0.424
+ recent history	0.297	0.276	0.461	0.468

origin, the flight number's delay history and the departure hour highest.

IX. WHAT-IF RESCHEDULING

Rule. Every December flight with predicted risk ≥ 0.5 is offered a move of $-2, -1, +1$ or $+2$ hours on the same day. For each candidate we recompute the hour-dependent features (time encodings, overnight flag, origin-hour congestion including the moved flight, origin-hour history) and score it with LR (the best model on validation) using rolling calibration. A flight takes its best candidate if risk drops by at least 0.05 and the target origin-hour slot has received fewer than three moved flights; the riskiest flights choose first.

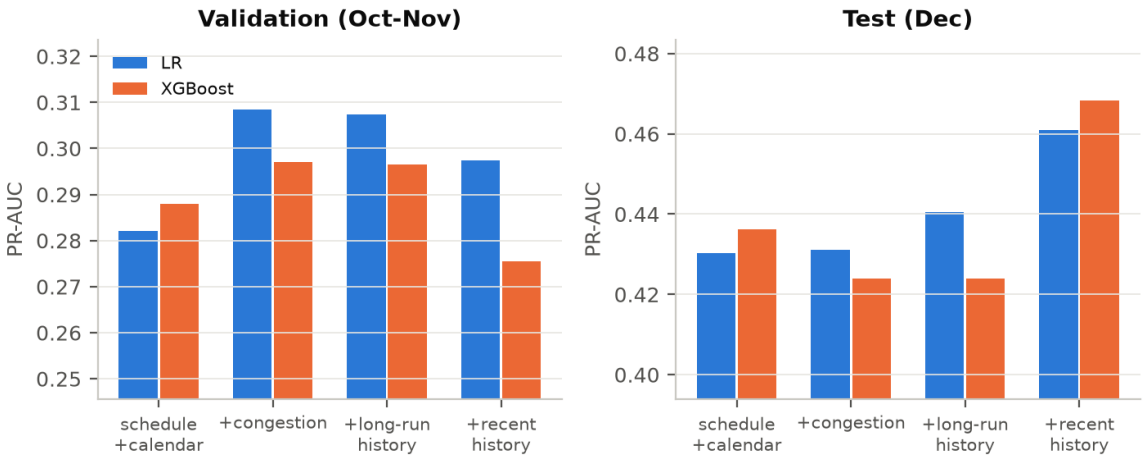


Fig. 5. PR-AUC as feature groups are added (left: validation, right: test).

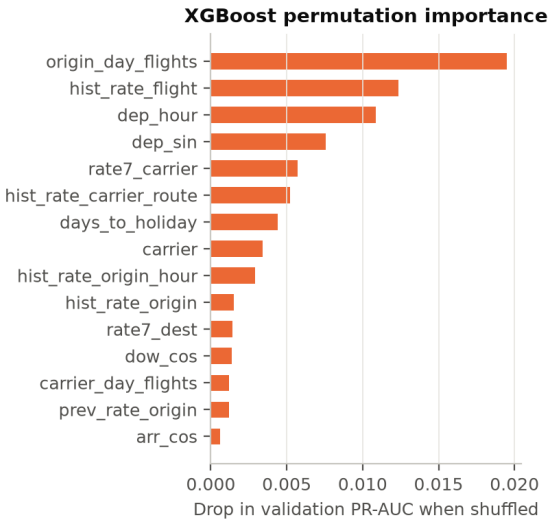


Fig. 6. Top-15 features by drop in validation PR-AUC when permuted (XGBoost, 3 repeats).

Assumptions. (i) The model's risk for the new slot is what the flight would face there; the model is associational, so this is a what-if estimate, not a causal effect. (ii) Aircraft rotations, crews, gates and passenger connections are ignored. (iii) Flights only move into hours in which the airport already operates that day, between 05:00 and 23:59.

Results. At threshold 0.5, 1,979 flights are flagged (their observed delay rate is 0.520). Only 133 have a move that clears the 0.05 gain, and 98 of these move earlier; their mean risk drops from 0.571 to 0.510. Summed over all flagged flights, expected delays fall from 1123 to 1115 (Table VII, Fig. 7). The model's time-of-day profile follows the shape of the observed December profile, so the direction of the recommendation (earlier is better) is plausible, but within ± 2 hours the predicted benefit is small. Much of the risk is attached to the day and the route rather than the hour.

TABLE VII
WHAT-IF RESCHEDULING ON DECEMBER (SENSITIVITY TO THE RISK THRESHOLD)

Risk threshold	0.4	0.5	0.6
Flagged flights	5750	1979	473
Observed delay rate of flagged	0.474	0.520	0.556
Flights moved	295	133	33
Mean risk before (moved)	0.504	0.571	0.657
Mean risk after (moved)	0.444	0.510	0.595
Expected delays, flagged, before	2796.9	1123.4	309.1
Expected delays, flagged, after	2779.1	1115.3	307.1

X. LIMITATIONS

No weather forecast is available, and the day-level weather explains much of the variance; this caps discrimination for any day-ahead model on this data. The data cover one year and only NYC departures, so inbound legs of an aircraft's rotation are invisible, and seasonal effects for the test months cannot be learned. Cancelled and diverted flights are dropped; they cluster on the worst days, so the labelled data slightly understate disruption. The previous-day outcomes are assumed known at the decision time. The rescheduling analysis is associational and ignores operational constraints.

XI. CONCLUSION

A strictly day-ahead delay model on the 2013 NYC flights reaches a December PR-AUC of about 0.46–0.47 against a base rate of 0.335, with LR and XGBoost close to each other once XGBoost is regularised to stop memorising training days. Calibration is the part that breaks under temporal shift: a calibrator fitted on calm months under-predicts a bad month, and a rolling recalibration based only on past outcomes largely repairs it. A what-if rescheduling rule finds that moving high-risk flights earlier is the only consistent recommendation, with small predicted gains. Phase 2 will compare these models with a compact MLP that learns embeddings for carrier, origin and destination on the same split and features.

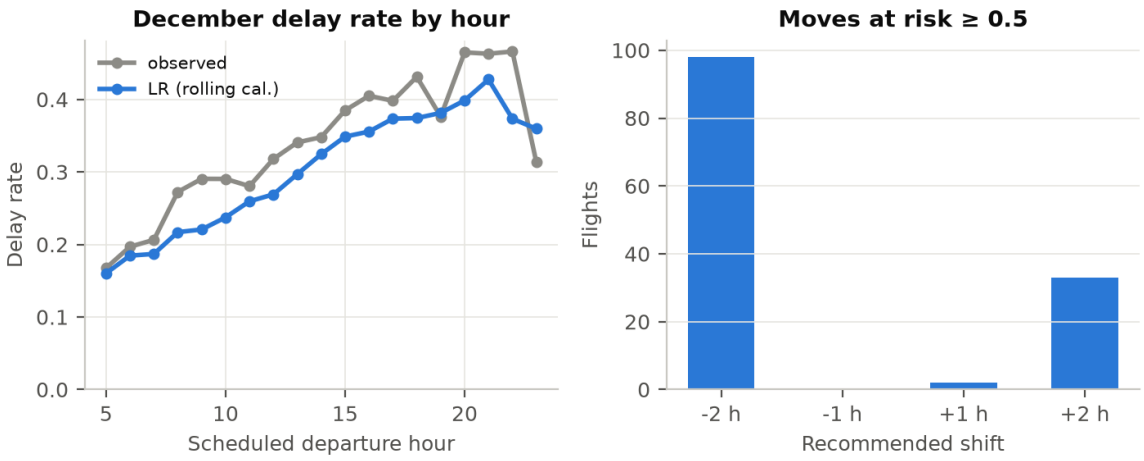


Fig. 7. Left: observed and predicted December delay rate by departure hour. Right: recommended shifts at risk threshold 0.5.

Team Contributions

The project was divided into five work packages, one per notebook, and the notebooks pass their outputs to one another in order. The report, slides and README were shared out as listed below.

- **Atanu Adhikari** (exploratory data analysis and presentation). Audited data quality (cancelled and diverted flights, missing tail numbers, duplicate ids), fixed the delay target and identified the post-departure columns that must be excluded, and analysed seasonality, time of day, day of week, carriers, routes and day-to-day persistence. This work underlies the Dataset and Exploratory Analysis section, including the base-rate differences between the training, validation and test months. Also prepared the presentation slides. Notebook: `01_eda`.
- **Sambhav Kumar** (feature engineering and README). Built the 35 schedule, calendar, congestion and history features, including the past-only aggregation, the Beta-prior smoothing and the cyclic time encodings, and wrote the leakage test that rebuilds the features after corrupting all outcomes from a cutoff date, together with the check that a deliberately leaky version is detected. This work underlies the Problem Formulation and Feature Engineering sections. Also wrote the repository README. Notebook: `02_feature_engineering`.
- **Nishant Ranjan Singh** (model training and report). Tuned logistic regression and XGBoost, diagnosed the overfitting of XGBoost on training days, chose the calibration method (Platt versus isotonic) and the rolling-calibration window on validation data, ran the class-weighting experiment and the feature-group ablation, and saved the trained models. This work underlies the Methods and Experimental Setup sections and the Overfitting and Class Imbalance results. Co-wrote this report. Notebook: `03_model_training`.
- **Sankalp M Tellur** (evaluation and calibration). Scored all models on the December test set (PR-AUC, ROC-AUC, Brier score, ECE), produced the precision–recall, ROC and reliability plots, set the operating point from the validation threshold, computed permutation importance, and generated the LaTeX tables and number macros that this report reads. This work underlies the Discrimination, Operating Point, Calibration and Feature Ablation and Importance results. Notebook: `04_evaluation_calibration`.
- **Prince Kumar Singh** (rescheduling and report). Designed and implemented the what-if rescheduling analysis: candidate moves of ± 1 and ± 2 hours with recomputed hour-dependent features, the gain and slot-capacity rules, the stated assumptions, and the sensitivity to the risk threshold. This work underlies the What-If Rescheduling section. Co-wrote this report. Notebook: `05_rescheduling`.

References

[1] M. Ball, C. Barnhart, M. Dresner, M. Hansen, K. Neels, A. Odoni, E. Peterson, L. Sherry, A. Trani, and B. Zou, "Total delay impact study: A comprehensive assessment of the costs and impacts of flight delay in the united states," NEXTOR, Tech. Rep., 2010.

[2] A. Sternberg, J. Soares, D. Carvalho, and E. Ogasawara, "A review on flight delay prediction," *arXiv preprint arXiv:1703.06118*, 2017.

[3] J. J. Rebollo and H. Balakrishnan, "Characterization and prediction of air traffic delays," *Transportation Research Part C: Emerging Technologies*, vol. 44, pp. 231–241, 2014.

[4] S. Choi, Y. J. Kim, S. Briceno, and D. Mavris, "Prediction of weather-induced airline delays based on machine learning algorithms," in *Proc. IEEE/AIAA 35th Digital Avionics Systems Conference (DASC)*, 2016.

[5] G. Gui, F. Liu, J. Sun, J. Yang, Z. Zhou, and D. Zhao, "Flight delay prediction based on aviation big data and machine learning," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 1, pp. 140–150, 2020.

[6] M. F. Yazdi, S. R. Kamel, S. J. M. Chabok, and M. Kheirabadi, "Flight delay prediction based on deep learning and Levenberg-Marquardt algorithm," *Journal of Big Data*, vol. 7, p. 106, 2020.

[7] N. Pyrgiotis, K. M. Malone, and A. Odoni, "Modelling delay propagation within an airport network," *Transportation Research Part C: Emerging Technologies*, vol. 27, pp. 60–75, 2013.

[8] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[10] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*. MIT Press, 1999, pp. 61–74.
[11] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proc. 8th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2002, pp. 694–699.
[12] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd Int. Conf. Machine Learning (ICML)*, 2005, pp. 625–632.
[13] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.
[14] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002.
[15] Z. C. Lipton, Y.-X. Wang, and A. J. Smola, "Detecting and correcting for label shift with black box predictors," in *Proc. 35th Int. Conf. Machine Learning (ICML)*, 2018, pp. 3122–3130.
[16] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 44:1–44:37, 2014.
[17] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015.
[18] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.
[19] H. Wickham, "nycflights13: Flights that departed NYC in 2013," R package, https://github.com/tidyverse/nycflights13, 2018.
[20] U.S. Bureau of Transportation Statistics, "Airline on-time performance data and documentation," https://www.transtats.bts.gov/, accessed Oct. 2026.
[21] D. Micci-Barreca, "A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems," *ACM SIGKDD Explorations Newsletter*, vol. 3, no. 1, pp. 27–32, 2001.
[22] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
[23] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.